

Chapitre I: La résolution des problèmes par la recherche

3eme ING IA/GL

Belabed Amine

Université De Tlemcen – Faculté des sciences, Département d'Informatique

2025-2026

Introduction

- **Un problème** : une situation devant laquelle on ne voit pas directement les étapes permettant de dépasser la situation et obtenir la solution.

Introduction

- **Un problème** : une situation devant laquelle on ne voit pas directement les étapes permettant de dépasser la situation et obtenir la solution.
- **Résoudre un problème** : revient à chercher un chemin qui mène d'une situation initiale à une autre finale pour atteindre un but.

Introduction

- **Un problème** : une situation devant laquelle on ne voit pas directement les étapes permettant de dépasser la situation et obtenir la solution.
- **Résoudre un problème** : revient à chercher un chemin qui mène d'une situation initiale à une autre finale pour atteindre un but.
- **Exemples** : les jeux, planification , la preuve de théorèmes , ...

Introduction

- **Un problème** : une situation devant laquelle on ne voit pas directement les étapes permettant de dépasser la situation et obtenir la solution.
- **Résoudre un problème** : revient à chercher un chemin qui mène d'une situation initiale à une autre finale pour atteindre un but.
- **Exemples** : les jeux, planification , la preuve de théorèmes , ...
- **La démarche d'IA** :

Introduction

- **Un problème** : une situation devant laquelle on ne voit pas directement les étapes permettant de dépasser la situation et obtenir la solution.
- **Résoudre un problème** : revient à chercher un chemin qui mène d'une situation initiale à une autre finale pour atteindre un but.
- **Exemples** : les jeux, planification , la preuve de théorèmes , ...
- **La démarche d'IA** :
 - ▶ L'algorithme doit être "neutre" sur le domaine concerné

Introduction

- **Un problème** : une situation devant laquelle on ne voit pas directement les étapes permettant de dépasser la situation et obtenir la solution.
- **Résoudre un problème** : revient à chercher un chemin qui mène d'une situation initiale à une autre finale pour atteindre un but.
- **Exemples** : les jeux, planification , la preuve de théorèmes , ...
- **La démarche d'IA** :
 - ▶ L'algorithme doit être "neutre" sur le domaine concerné
 - ▶ Les connaissances de description du problème et de sa résolution doivent être clairement séparés de l'algorithme.

Comment définir un problème ?

Un problème peut être défini par 5 composants :

- 1 Définir l'espace des états.

Comment définir un problème ?

Un problème peut être défini par 5 composants :

- 1 Définir l'espace des états.
- 2 Définir l'état initial : l'état de départ ;

Comment définir un problème ?

Un problème peut être défini par 5 composants :

- ① Définir l'espace des états.
- ② Définir l'état initial : l'état de départ ;
- ③ Définir les opérateurs

Comment définir un problème ?

Un problème peut être défini par 5 composants :

- ① Définir l'espace des états.
- ② Définir l'état initial : l'état de départ ;
- ③ Définir les opérateurs
 - ▶ Fonction successeur : permettant de retourner l'ensemble des états atteignables par une action particulière.

Comment définir un problème ?

Un problème peut être défini par 5 composants :

- ① Définir l'espace des états.
- ② Définir l'état initial : l'état de départ ;
- ③ Définir les opérateurs
 - ▶ Fonction successeur : permettant de retourner l'ensemble des états atteignables par une action particulière.
 - ▶ L'état initial + l'ensemble des états atteignables : l'espace des états.

Comment définir un problème ?

Un problème peut être défini par 5 composants :

- ① Définir l'espace des états.
- ② Définir l'état initial : l'état de départ ;
- ③ Définir les opérateurs
 - ▶ **Fonction successeur** : permettant de retourner l'ensemble des états **atteignables** par une action particulière.
 - ▶ L'état initial + l'ensemble des états atteignables : l'espace des états.
 - ▶ Un chemin dans l'espace des états est une séquence d'états connectés par une séquence d'actions (opérations).

Comment définir un problème ?

Un problème peut être défini par 5 composants :

- ① Définir l'espace des états.
- ② Définir l'état initial : l'état de départ ;
- ③ Définir les opérateurs
 - ▶ Fonction successeur : permettant de retourner l'ensemble des états atteignables par une action particulière.
 - ▶ L'état initial + l'ensemble des états atteignables : l'espace des états.
 - ▶ Un chemin dans l'espace des états est une séquence d'états connectés par une séquence d'actions (opérations).
- ④ Une fonction de test : permettant de savoir que la solution est trouvée.

Comment définir un problème ?

Un problème peut être défini par 5 composants :

- ① Définir l'espace des états.
- ② Définir l'état initial : l'état de départ ;
- ③ Définir les opérateurs
 - ▶ **Fonction successeur** : permettant de retourner l'ensemble des états **atteignables** par une action particulière.
 - ▶ L'état initial + l'ensemble des états atteignables : l'espace des états.
 - ▶ Un chemin dans l'espace des états est une séquence d'états connectés par une séquence d'actions (opérations).
- ④ Une fonction de test : permettant de savoir que la solution est trouvée.
- ⑤ Une fonction de coût : Associe un coût au chemin de la solution.

Comment définir un problème ?

Un problème peut être défini par 5 composants :

- ① Définir l'espace des états.
- ② Définir l'état initial : l'état de départ ;
- ③ Définir les opérateurs
 - ▶ **Fonction successeur** : permettant de retourner l'ensemble des états **atteignables** par une action particulière.
 - ▶ L'état initial + l'ensemble des états atteignables : l'espace des états.
 - ▶ Un chemin dans l'espace des états est une séquence d'états connectés par une séquence d'actions (opérations).
- ④ Une **fonction de test** : permettant de savoir que la solution est trouvée.
- ⑤ Une **fonction de coût** : Associe un coût au chemin de la solution.
 - ▶ Une **solution optimale** est une solution qui minimise la fonction de coût.

Comment définir un problème ?

Exemple : Le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- L'espace des états et état initial : ?
- Les Actions (opérateurs) ?
- Test du but ?
- Le coût d'une Action ?

Comment définir un problème ?

Exemple : Le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- L'espace des états et état initial : chaque état est une disposition de chiffres.

Comment définir un problème ?

Exemple : Le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- **L'espace des états et état initial** : chaque état est une disposition de chiffres.
- **Les Actions (opérateurs)** : déplacer le blanc à gauche, à droite, en haut ou en bas.

Comment définir un problème ?

Exemple : Le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- **L'espace des états et état initial** : chaque état est une disposition de chiffres.
- **Les Actions (opérateurs)** : déplacer le blanc à gauche, à droite, en haut ou en bas.
- **Test du but** : l'état final est donné.

Comment définir un problème ?

Exemple : Le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

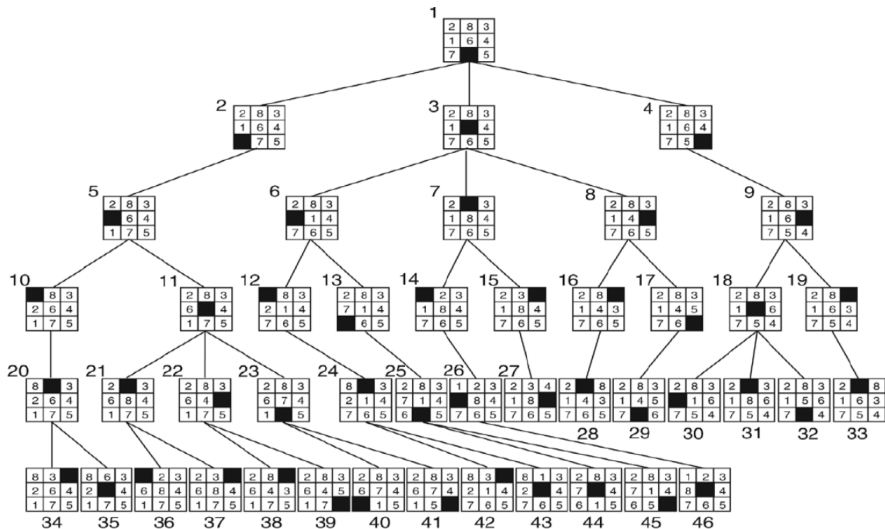
1	2	3
4	5	6
7	8	

Etat final

- **L'espace des états et état initial** : chaque état est une disposition de chiffres.
- **Les Actions (opérateurs)** : déplacer le blanc à gauche, à droite, en haut ou en bas.
- **Test du but** : l'état final est donné.
- **Le coût d'une Action** : 1 / mouvement.

Comment définir un problème ?

Exemple : Le jeu de taquin



Comment définir un problème ?

Exemple : Le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- 8-puzzle a $9!$ états possibles.

Comment définir un problème ?

Exemple : Le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- 8-puzzle a $9!$ états possibles.
- 15-puzzle a environ de $\simeq 10^{13}$ états.

Comment définir un problème ?

Exemple : Le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- 8-puzzle a $9!$ états possibles.
- 15-puzzle a environ de $\simeq 10^{13}$ états.
- 24-puzzle a $\simeq 10^{25}$ états!!!

Critères d'évaluation d'un algorithme de résolution

- **La complétude** : lorsque la méthode garantit de trouver une solution si elle existe ;

Critères d'évaluation d'un algorithme de résolution

- **La complétude** : lorsque la méthode garantit de trouver une solution si elle existe ;
- **L'optimalité** : ce critère concerne l'optimalité de la solution trouvée, en se basant sur une fonction de coût ;

Critères d'évaluation d'un algorithme de résolution

- **La complétude** : lorsque la méthode garantit de trouver une solution si elle existe ;
- **L'optimalité** : ce critère concerne l'optimalité de la solution trouvée, en se basant sur une fonction de coût ;
- **La complexité en temps** : Combien de temps faut-il pour trouver une solution ;

Critères d'évaluation d'un algorithme de résolution

- **La complétude** : lorsque la méthode garantit de trouver une solution si elle existe ;
- **L'optimalité** : ce critère concerne l'optimalité de la solution trouvée, en se basant sur une fonction de coût ;
- **La complexité en temps** : Combien de temps faut-il pour trouver une solution ;
- **La complexité en espace** : combien l'algorithme utilise de la mémoire pour trouver une solution.

Critères d'évaluation d'un algorithme de résolution

La complexité est exprimée en utilisant la fonction **Big O (Borne supérieure asymptotique)**, avec les quantités suivantes :

- **b** , le facteur de branchement : le nombre maximum de successeurs à un nœud ; si elle existe ;

Critères d'évaluation d'un algorithme de résolution

La complexité est exprimée en utilisant la fonction **Big O (Borne supérieure asymptotique)**, avec les quantités suivantes :

- **b** , le facteur de branchement : le nombre maximum de successeurs à un nœud ; si elle existe ;
- **d** , la profondeur du nœud but le moins profond (profondeur de la solution) ;

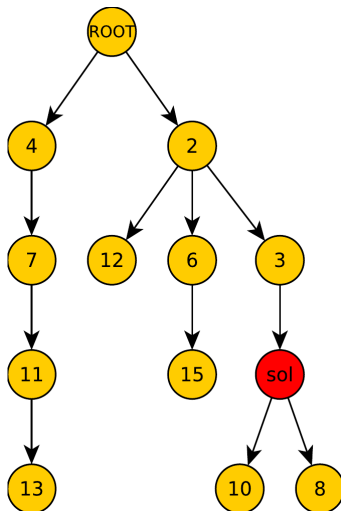
Critères d'évaluation d'un algorithme de résolution

La complexité est exprimée en utilisant la fonction **Big O (Borne supérieure asymptotique)**, avec les quantités suivantes :

- **b** , le facteur de branchement : le nombre maximum de successeurs à un nœud ; si elle existe ;
- **d** , la profondeur du nœud but le moins profond (profondeur de la solution) ;
- **m** : la longueur maximale d'un chemin dans l'espace d'états (profondeur maximum de l'arbre de recherche).

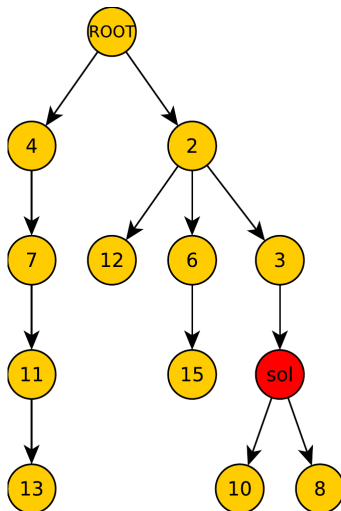
Critères d'évaluation d'un algorithme de résolution

- Exemple :



Critères d'évaluation d'un algorithme de résolution

- Exemple :



- $(b=3)$; $(d=3)$; $(m=4)$.

Critères d'évaluation d'un algorithme de résolution

→ Le Big O (Borne supérieure asymptotique)

- Définit la limite supérieure (en temps ou en espace) d'un algorithme.

Big O

On dit qu'une fonction f est un grand O d'une fonction g si et seulement si : $\exists c > 0, \exists n_0 > 0$ tel que $\forall n > n_0, f(n) < c \times g(n)$

● Exemples :

- ▶ Si $f(n) = 4$ alors $f(n) = O(1)$, prendre par exemple $c=5$ et $n_0=0$.
- ▶ Si $f(n) = 3n + 2$ alors $f(n) = O(n)$, prendre par exemple $c=4$ et $n_0=2$.
- ▶ Si $f(n) = n^2 + 3$ alors $f(n) = O(n^2)$, prendre par exemple $c=3$ et $n_0=1$.

Critères d'évaluation d'un algorithme de résolution

→ Le Big O (Borne supérieure asymptotique)

- Définit la limite supérieure (en temps ou en espace) d'un algorithme.
- Définit la grandeur d'une fonction (comportement à l'infini).

Big O

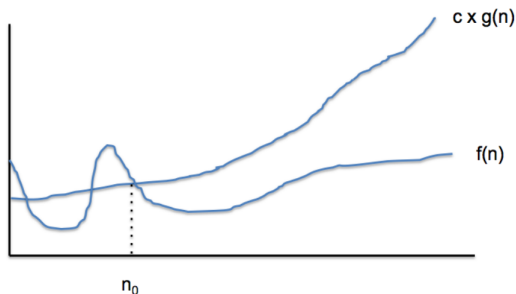
On dit qu'une fonction f est un grand O d'une fonction g si et seulement si : $\exists c > 0, \exists n_0 > 0$ tel que $\forall n > n_0, f(n) < c \times g(n)$

• Exemples :

- ▶ Si $f(n) = 4$ alors $f(n) = O(1)$, prendre par exemple $c=5$ et $n_0=0$.
- ▶ Si $f(n) = 3n + 2$ alors $f(n) = O(n)$, prendre par exemple $c=4$ et $n_0=2$.
- ▶ Si $f(n) = n^2 + 3$ alors $f(n) = O(n^2)$, prendre par exemple $c=3$ et $n_0=1$.

Critères d'évaluation d'un algorithme de résolution

→ Le Big O (Borne supérieure asymptotique)



- Complexité de quelques algorithmes :

- ▶ Accès à un tableau : $O(1)$, complexité constante.
- ▶ Recherche dichotomique : $O(\log(n))$, complexité logarithmique.
- ▶ Parcours d'une liste : $O(n)$, complexité linéaire.
- ▶ Parcours de tableau 2D : $O(n^2)$, complexité quadratique (polynomiale).

Stratégies de recherche d'une solution

Deux types de stratégies ont été développés :

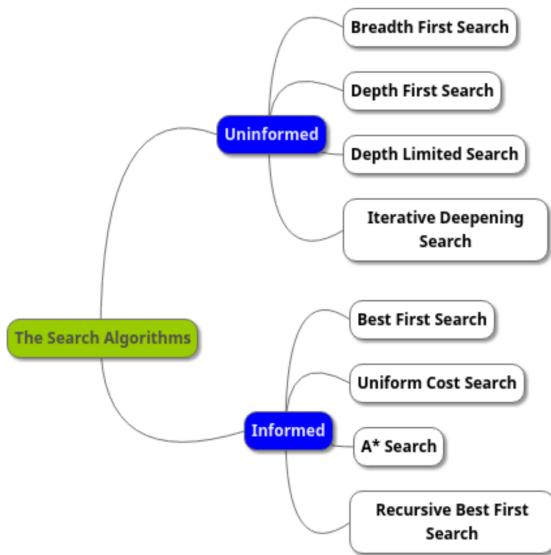
- **L'exploration non informée** : ne dispose pas d'information sur les états. Elle peut générer des successeurs et distinguer un état final d'un état non final.

Stratégies de recherche d'une solution

Deux types de stratégies ont été développés :

- **L'exploration non informée** : ne dispose pas d'information sur les états. Elle peut générer des successeurs et distinguer un état final d'un état non final.
- **L'exploration informée** : elle est capable de déterminer si un état non final est plus prometteur qu'un autre.

Stratégies de recherche d'une solution



Source : <http://teleported.in/posts/ai-search-algorithms/>

Les algorithmes de recherche non informés

Les algorithmes de recherche non informés

- ➊ Recherche en largeur d'abord (**Breadth-first search, BFS**).

Les algorithmes de recherche non informés

- ➊ Recherche en largeur d'abord (**Breadth-first search, BFS**).
- ➋ Recherche en profondeur d'abord (**Depth-first search, DFS**).

Les algorithmes de recherche non informés

- ① Recherche en largeur d'abord (**Breadth-first search, BFS**).
- ② Recherche en profondeur d'abord (**Depth-first search, DFS**).
- ③ Recherche en profondeur limitée (**Depth-limited search, DLS**).

Les algorithmes de recherche non informés

- ➊ Recherche en largeur d'abord (**Breadth-first search, BFS**).
- ➋ Recherche en profondeur d'abord (**Depth-first search, DFS**).
- ➌ Recherche en profondeur limitée (**Depth-limited search, DLS**).
- ➍ Recherche en profondeur itérative (**Iterative deepening depth-first search, IDS**).

Les algorithmes de recherche non informés

- ➊ Recherche en largeur d'abord (**Breadth-first search, BFS**).
- ➋ Recherche en profondeur d'abord (**Depth-first search, DFS**).
- ➌ Recherche en profondeur limitée (**Depth-limited search, DLS**).
- ➍ Recherche en profondeur itérative (**Iterative deepening depth-first search, IDS**).
- ➎ Recherche à cout uniforme (**Uniform-cost search UCS**).

Structure générale d'un algorithme de recherche

Set $Open = \{E_0\};$

Set $Closed = \{\};$

repeat

if $Open = \{\}$ **then**

 fin avec echec;

else

 choisir un noeud n de **Open** (supprimer de **Open** save to

Closed); **if** $n \in \textit{But}$ (*état final*) **then**

 fin avec succès ;

else

 générer les successeurs m de n (utilisation des opérations);

 Insérer les dans **Open** (seulement si $m \notin \{Open \cup Closed\}$

);

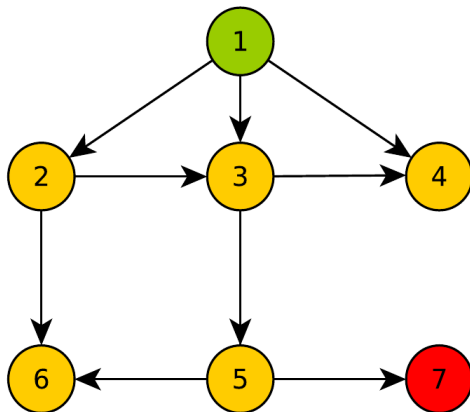
end

end

until:

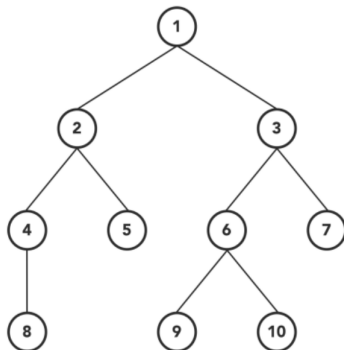
Structure générale d'un algorithme de recherche

- **Exemple** : Open=Pile.



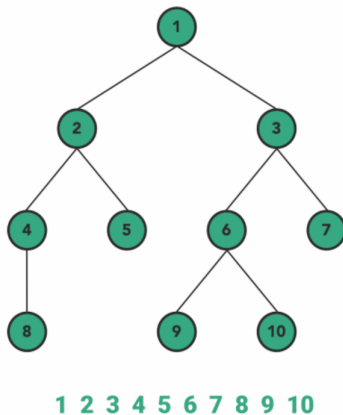
Recherche en largeur d'abord (Breadth-first search, BFS).

- **Principe** : On examine d'abord l'état initial, puis ses successeurs, puis les successeurs des successeurs, etc.



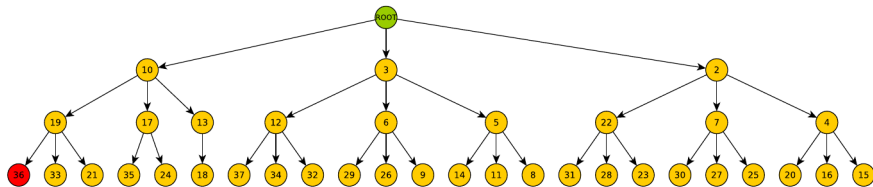
Recherche en largeur d'abord (Breadth-first search, BFS).

- **Principe** : On examine d'abord l'état initial, puis ses successeurs, puis les successeurs des successeurs, etc.



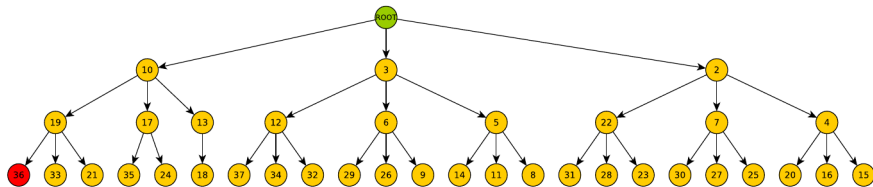
Recherche en largeur d'abord (Breadth-first search, BFS).

- Analyse de l'algorithme de recherche en largeur d'abord :



Recherche en largeur d'abord (Breadth-first search, BFS).

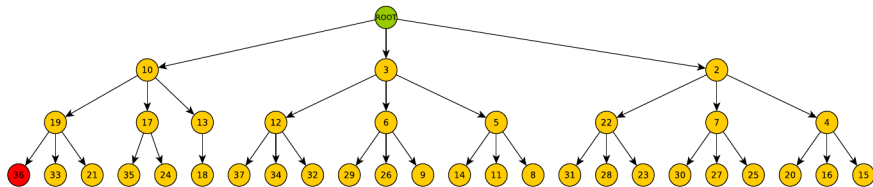
- Analyse de l'algorithme de recherche en largeur d'abord :



- Complétude** : oui pour les espaces d'états finis et acycliques

Recherche en largeur d'abord (Breadth-first search, BFS).

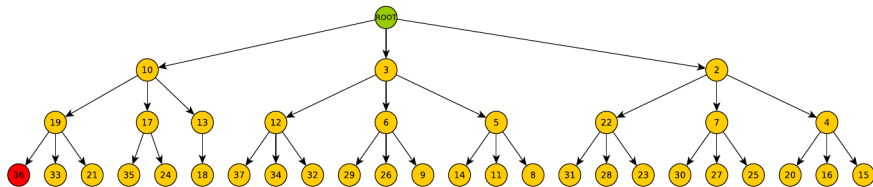
- Analyse de l'algorithme de recherche en largeur d'abord :



- **Complétude** : oui pour les espaces d'états finis et acycliques
- **Complexité en temps** : exponentielle, $O(b^d)$ ($O(b^{d+1})$) où d est la profondeur et b est le facteur de branchement.

Recherche en largeur d'abord (Breadth-first search, BFS).

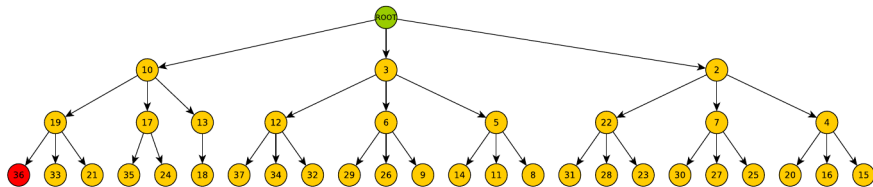
- Analyse de l'algorithme de recherche en largeur d'abord :



- Complétude** : oui pour les espaces d'états finis et acycliques
- Complexité en temps** : exponentielle, $O(b^d)$ ($O(b^{d+1})$) où d est la profondeur et b est le facteur de branchement.
- Complexité en espace** : exponentielle, $O(b^d)$.

Recherche en largeur d'abord (Breadth-first search, BFS).

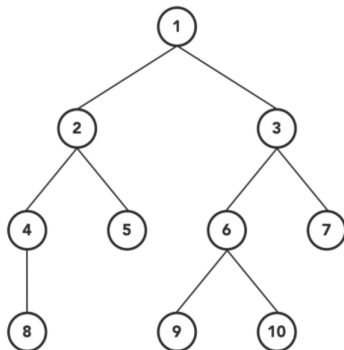
- Analyse de l'algorithme de recherche en largeur d'abord :



- **Complétude** : oui pour les espaces d'états finis et acycliques
- **Complexité en temps** : exponentielle, $O(b^d)$ ($O(b^{d+1})$) où d est la profondeur et b est le facteur de branchement.
- **Complexité en espace** : exponentielle, $O(b^d)$.
- **Optimalité** : non.

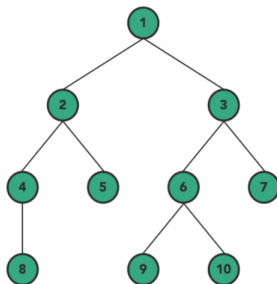
Recherche en profondeur d'abord (Depth-first search, DFS)

- **Principe** : On examine d'abord l'état initial, puis le premier fils, puis le fils du fils, etc.



Recherche en profondeur d'abord (Depth-first search, DFS)

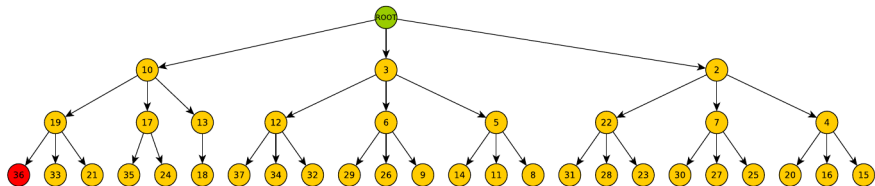
- **Principe** : On examine d'abord l'état initial, puis le premier fils, puis le fils du fils, etc.



1 2 4 8 5 3 6 9 10 7

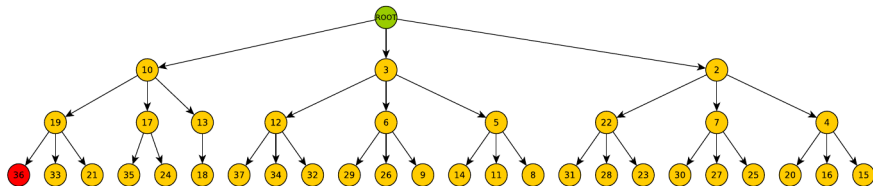
Recherche en profondeur d'abord (Depth-first search, DFS)

- Analyse de l'algorithme :



Recherche en profondeur d'abord (Depth-first search, DFS)

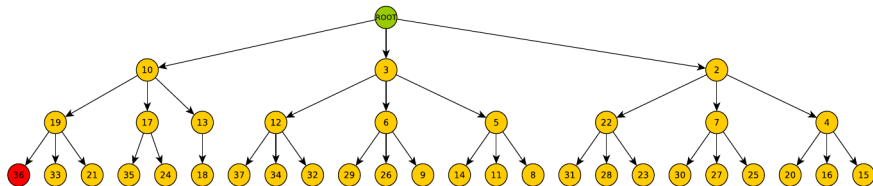
- Analyse de l'algorithme :



- Complétude** : oui pour les espaces d'états finis et acycliques

Recherche en profondeur d'abord (Depth-first search, DFS)

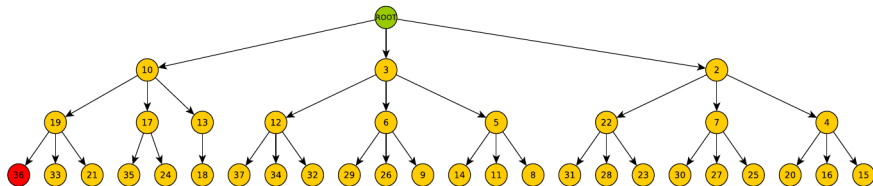
- Analyse de l'algorithme :



- **Complétude** : oui pour les espaces d'états finis et acycliques
- **Complexité en temps** : exponentielle, $O(b^m)$.

Recherche en profondeur d'abord (Depth-first search, DFS)

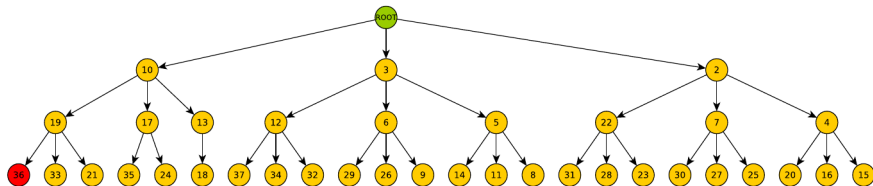
- Analyse de l'algorithme :



- **Complétude** : oui pour les espaces d'états finis et acycliques
- **Complexité en temps** : exponentielle, $O(b^m)$.
- **Complexité en espace** : $O(b * m)$ (linéaire).

Recherche en profondeur d'abord (Depth-first search, DFS)

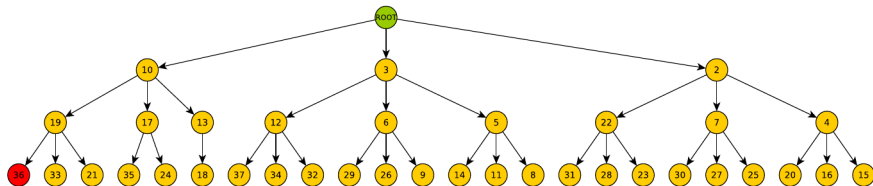
- Analyse de l'algorithme :



- **Complétude** : oui pour les espaces d'états finis et acycliques
- **Complexité en temps** : exponentielle, $O(b^m)$.
- **Complexité en espace** : $O(b * m)$ (linéaire).
- **Optimalité** : non.

Recherche en profondeur d'abord (Depth-first search, DFS)

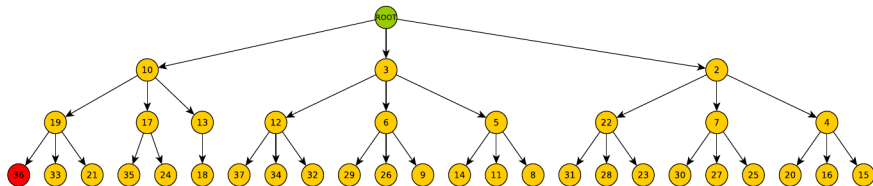
- Analyse de l'algorithme :



- **Complétude** : oui pour les espaces d'états finis et acycliques
- **Complexité en temps** : exponentielle, $O(b^m)$.
- **Complexité en espace** : $O(b * m)$ (linéaire).
- **Optimalité** : non.
- **Discussion** : pour $b=10$, $d=m=12$ et 100 octets/noeud :

Recherche en profondeur d'abord (Depth-first search, DFS)

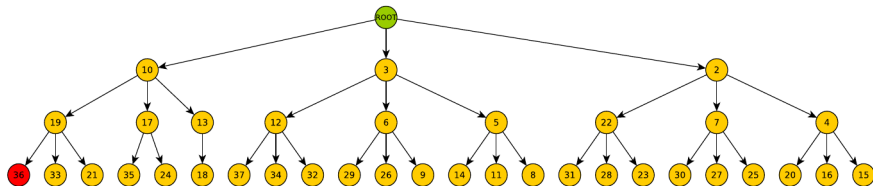
- Analyse de l'algorithme :



- **Complétude** : oui pour les espaces d'états finis et acycliques
- **Complexité en temps** : exponentielle, $O(b^m)$.
- **Complexité en espace** : $O(b * m)$ (linéaire).
- **Optimalité** : non.
- **Discussion** : pour $b=10$, $d=m=12$ et 100 octets/noeud :
 - Recherche en profondeur : 12 Ko.

Recherche en profondeur d'abord (Depth-first search, DFS)

- Analyse de l'algorithme :



- **Complétude** : oui pour les espaces d'états finis et acycliques
- **Complexité en temps** : exponentielle, $O(b^m)$.
- **Complexité en espace** : $O(b * m)$ (linéaire).
- **Optimalité** : non.
- **Discussion** : pour $b=10$, $d=m=12$ et 100 octets/noeud :
 - ▶ Recherche en profondeur : 12 Ko.
 - ▶ Recherche en largeur 111 Tera-octets.

Recherche en profondeur limitée (Depth-limited search, DLS)

- **Principe** : algorithme de recherche en profondeur avec une limite de profondeur d'exploration L .
 - ▶ Les noeuds de profondeur L n'ont pas de successeurs

Analyse de l'algorithme :

- **Complétude** : Oui si $L \geq d$

Recherche en profondeur limitée (Depth-limited search, DLS)

- **Principe** : algorithme de recherche en profondeur avec une limite de profondeur d'exploration L .
 - ▶ Les noeuds de profondeur L n'ont pas de successeurs

Analyse de l'algorithme :

- **Complétude** : Oui si $L \geq d$
- **Complexité en temps** : exponentielle, $O(b^L)$.

Recherche en profondeur limitée (Depth-limited search, DLS)

- **Principe** : algorithme de recherche en profondeur avec une limite de profondeur d'exploration L .
 - ▶ Les noeuds de profondeur L n'ont pas de successeurs

Analyse de l'algorithme :

- **Complétude** : Oui si $L \geq d$
- **Complexité en temps** : exponentielle, $O(b^L)$.
- **Complexité en espace** : $O(b * L)$ (linéaire).

Recherche en profondeur limitée (Depth-limited search, DLS)

- **Principe** : algorithme de recherche en profondeur avec une limite de profondeur d'exploration L .
 - ▶ Les noeuds de profondeur L n'ont pas de successeurs

Analyse de l'algorithme :

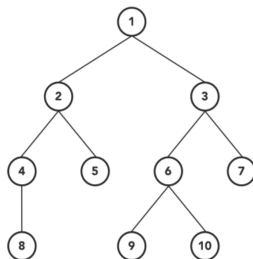
- **Complétude** : Oui si $L \geq d$
- **Complexité en temps** : exponentielle, $O(b^L)$.
- **Complexité en espace** : $O(b * L)$ (linéaire).
- **Optimalité** : non.

Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

- **Principe** : Approfondissement itératif = répéter DLS pour toutes les valeurs possibles de $L=0,1, 2\dots$ jusqu'à trouver une solution.

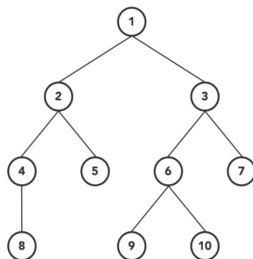
Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

- **Principe** : Approfondissement itératif = répéter DLS pour toutes les valeurs possibles de $L=0,1, 2\ldots$ jusqu'à trouver une solution.
- **Exemple** :



Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

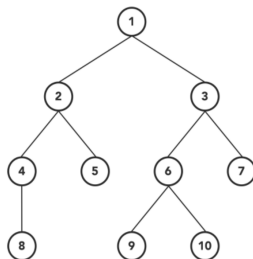
- **Principe** : Approfondissement itératif = répéter DLS pour toutes les valeurs possibles de $L=0,1, 2\dots$ jusqu'à trouver une solution.
- **Exemple** :



- **Ordre de visite** :

Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

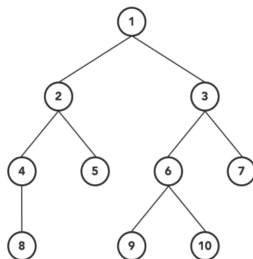
- **Principe** : Approfondissement itératif = répéter DLS pour toutes les valeurs possibles de $L=0,1, 2\dots$ jusqu'à trouver une solution.
- **Exemple** :



- **Ordre de visite** :
 - 1 $L=0$: 1.

Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

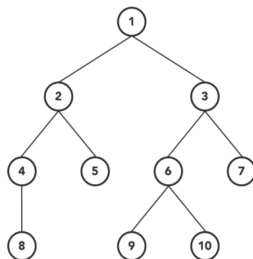
- **Principe** : Approfondissement itératif = répéter DLS pour toutes les valeurs possibles de $L=0,1, 2\dots$ jusqu'à trouver une solution.
- **Exemple** :



- **Ordre de visite** :
 - 1 $L=0$: 1.
 - 2 $L=1$: 1, 2, 3.

Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

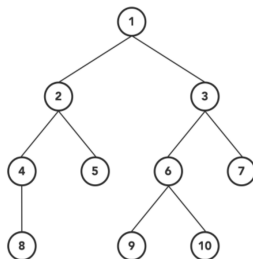
- **Principe** : Approfondissement itératif = répéter DLS pour toutes les valeurs possibles de $L=0,1, 2\dots$ jusqu'à trouver une solution.
- **Exemple** :



- **Ordre de visite** :
 - 1 $L=0$: 1.
 - 2 $L=1$: 1, 2, 3.
 - 3 $L=2$: 1, 2, 4, 5, 3, 6, 7.

Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

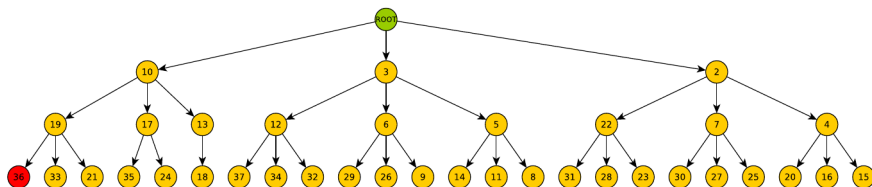
- **Principe** : Approfondissement itératif = répéter DLS pour toutes les valeurs possibles de $L=0,1, 2\dots$ jusqu'à trouver une solution.
- **Exemple** :



- **Ordre de visite** :
 - 1 $L=0$: 1.
 - 2 $L=1$: 1, 2, 3.
 - 3 $L=2$: 1, 2, 4, 5, 3, 6, 7.
 - 4 $L=3$: 1, 2, 4, 8, 5, 3, 6, 9, 10, 7.

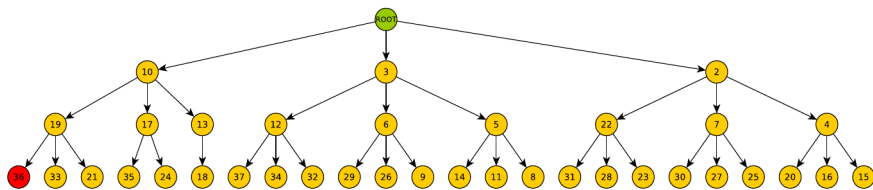
Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

- Analyse de l'algorithme :



Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

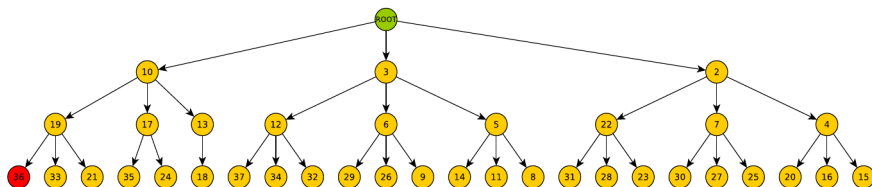
- Analyse de l'algorithme :



- Complétude : Oui.

Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

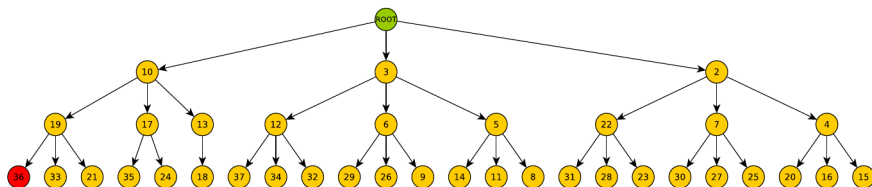
- Analyse de l'algorithme :



- Complétude : Oui.
- Complexité en temps : exponentielle, $O(b^d)$.

Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

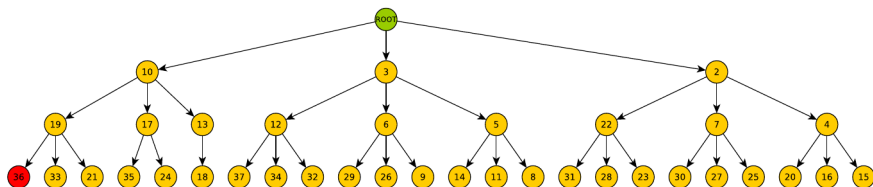
- Analyse de l'algorithme :



- Complétude : Oui.
- Complexité en temps : exponentielle, $O(b^d)$.
- Complexité en espace : linéaire, $O(b * d)$.

Recherche en profondeur itérative (Iterative deepening depth-first search, IDS)

- Analyse de l'algorithme :



- Complétude : Oui.
- Complexité en temps : exponentielle, $O(b^d)$.
- Complexité en espace : linéaire, $O(b * d)$.
- Optimalité : non (Oui si coût unitaire).

Recherche à coût uniforme (Uniform-Cost Search)

- **Principe** : la priorité est donnée aux nœuds ayant le coût le plus bas.

Recherche à coût uniforme (Uniform-Cost Search)

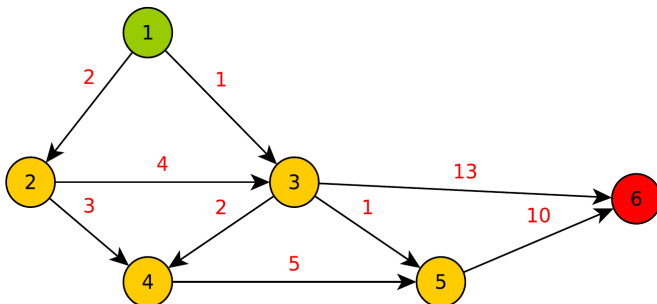
- **Principe** : la priorité est donnée aux nœuds ayant le coût le plus bas.
 - ▶ La structure de données utilisée est une file triée selon le coût.

Recherche à coût uniforme (Uniform-Cost Search)

- **Principe** : la priorité est donnée aux nœuds ayant le coût le plus bas.
 - ▶ La structure de données utilisée est une file triée selon le coût.
 - ▶ Est équivalente à la méthode largeur d'abord si le coût des actions est toujours le même.

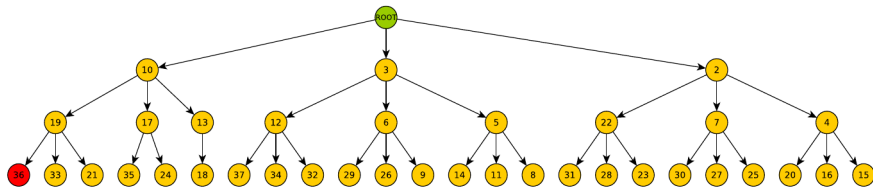
Recherche à coût uniforme (Uniform-Cost Search)

- **Principe** : la priorité est donnée aux nœuds ayant le coût le plus bas.
 - ▶ La structure de données utilisée est une file triée selon le coût.
 - ▶ Est équivalente à la méthode largeur d'abord si le coût des actions est toujours le même.
- **Exemple** :



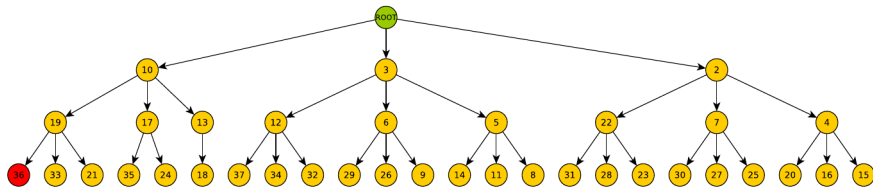
Recherche à coût uniforme (Uniform-Cost Search)

- Analyse de l'algorithme :



Recherche à coût uniforme (Uniform-Cost Search)

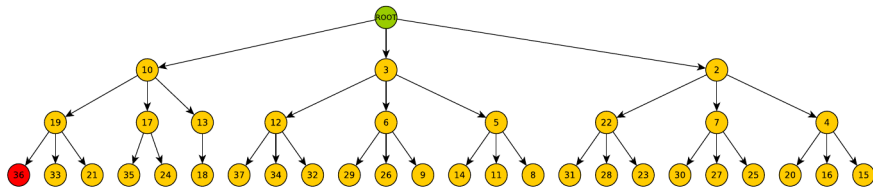
- Analyse de l'algorithme :



- Complétude : Oui.

Recherche à coût uniforme (Uniform-Cost Search)

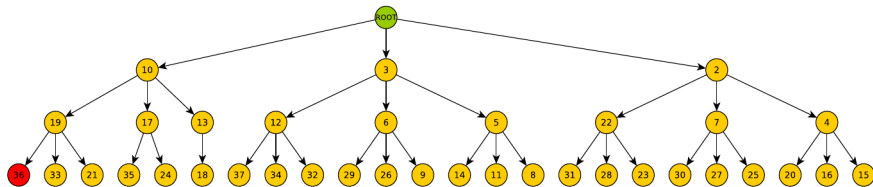
- Analyse de l'algorithme :



- Complétude : Oui.
- Complexité en temps : exponentielle, $(O(b^d))$, $O(b^{\frac{C^*}{\epsilon}})$.

Recherche à coût uniforme (Uniform-Cost Search)

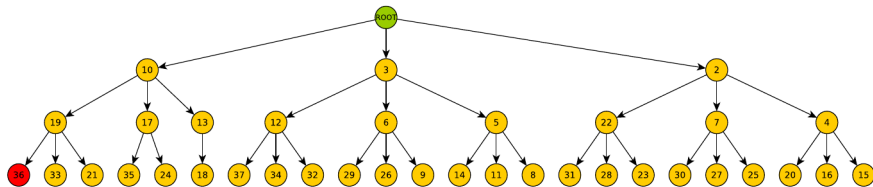
- Analyse de l'algorithme :



- **Complétude** : Oui.
- **Complexité en temps** : exponentielle, $(O(b^d))$, $O(b^{\frac{C^*}{\epsilon}})$.
 - C^* : le coût optimal d'une solution ; ϵ : le coût minimum.

Recherche à coût uniforme (Uniform-Cost Search)

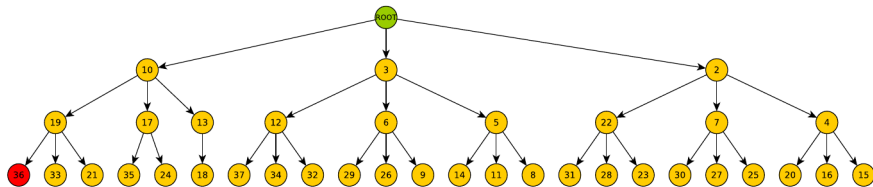
- Analyse de l'algorithme :



- **Complétude** : Oui.
- **Complexité en temps** : exponentielle, $(O(b^d))$, $O(b^{\frac{C^*}{\epsilon}})$.
 - C^* : le coût optimal d'une solution ; ϵ : le coût minimum.
- **Complexité en espace** : exponentielle , $O(b^d)$.

Recherche à coût uniforme (Uniform-Cost Search)

- Analyse de l'algorithme :



- **Complétude** : Oui.
- **Complexité en temps** : exponentielle, $(O(b^d))$, $O(b^{\frac{C^*}{\epsilon}})$.
 - C^* : le coût optimal d'une solution ; ϵ : le coût minimum.
- **Complexité en espace** : exponentielle , $O(b^d)$.
- **Optimalité** : Oui.

Les algorithmes de recherches informés (heuristiques)

La notion d'heuristique

- **Une heuristique** : un critère ou une méthode permettant de déterminer parmi plusieurs choix de solutions celle qui promet d'être la plus efficace pour atteindre un but.

La notion d'heuristique

- Une **heuristique** : un critère ou une méthode permettant de déterminer parmi plusieurs choix de solutions celle qui promet d'être la plus efficace pour atteindre un but.
- Une fonction (notée **h**) qui associer à chaque un état une **estimation du coût** pour atteindre l'état final a partir de cet état.

La notion d'heuristique

- Une **heuristique** : un critère ou une méthode permettant de déterminer parmi plusieurs choix de solutions celle qui promet d'être la plus efficace pour atteindre un but.
- Une fonction (notée **h**) qui associer à chaque un état une **estimation du coût** pour atteindre l'état final a partir de cet état.
- **Exemple** : pour le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

La notion d'heuristique

- Une **heuristique** : un critère ou une méthode permettant de déterminer parmi plusieurs choix de solutions celle qui promet d'être la plus efficace pour atteindre un but.
- Une fonction (notée ***h***) qui associer à chaque un état une **estimation du coût** pour atteindre l'état final a partir de cet état.
- **Exemple** : pour le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- Le nombre de plaquettes mal placées, $h(n) = ?$

La notion d'heuristique

- Une **heuristique** : un critère ou une méthode permettant de déterminer parmi plusieurs choix de solutions celle qui promet d'être la plus efficace pour atteindre un but.
- Une fonction (notée ***h***) qui associer à chaque un état une **estimation du coût** pour atteindre l'état final a partir de cet état.
- **Exemple** : pour le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- ▶ Le nombre de plaquettes mal placées, $h(n)=?$ ($h(n)=7$)
- ▶ La somme de distances de Manhattan (nombre de déplacement pour atteindre l'état final) : $h(n)=?$.

La notion d'heuristique

- Une **heuristique** : un critère ou une méthode permettant de déterminer parmi plusieurs choix de solutions celle qui promet d'être la plus efficace pour atteindre un but.
- Une fonction (notée ***h***) qui associer à chaque un état une **estimation du coût** pour atteindre l'état final a partir de cet état.
- **Exemple** : pour le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- ▶ Le nombre de plaquettes mal placées, $h(n)=?$ ($h(n)=7$)
- ▶ La somme de distances de Manhattan (nombre de déplacement pour atteindre l'état final) : $h(n)=?$.

La notion d'heuristique

- Une **heuristique** : un critère ou une méthode permettant de déterminer parmi plusieurs choix de solutions celle qui promet d'être la plus efficace pour atteindre un but.
- Une fonction (notée ***h***) qui associer à chaque un état une **estimation du coût** pour atteindre l'état final a partir de cet état.
- **Exemple** : pour le jeu de taquin

2	8	7
1	4	6
3		5

Etat N

1	2	3
4	5	6
7	8	

Etat final

- ▶ Le nombre de plaquettes mal placées, $h(n) = ?$ ($h(n) = 7$)
- ▶ La somme de distances de Manhattan (nombre de déplacement pour atteindre l'état final) : $h(n) = ?$. ($h(n) = 1 + 1 + 4 + 1 + 2 + 0 + 4 = 13$).

Heuristique : caractéristiques

- Une heuristique est dite **admissible** si elle ne surestime jamais le coût pour atteindre le but :

$$\forall n : h(n) \leq h^*(n)$$

Avec $h^*(n)$: le coût réel depuis l'état n jusqu'à l'état final.

Heuristique : caractéristiques

- Une heuristique est dite **admissible** si elle ne surestime jamais le coût pour atteindre le but :

$$\forall n : h(n) \leq h^*(n)$$

Avec $h^*(n)$: le coût réel depuis l'état n jusqu'à l'état final.

- Deux heuristiques admissibles h_1 et h_2 , on dit que h_1 **domine** h_2 si :

$$\forall n : h_1(n) \geq h_2(n)$$

Heuristique : caractéristiques

- Une heuristique est dite **admissible** si elle ne surestime jamais le coût pour atteindre le but :

$$\forall n : h(n) \leq h^*(n)$$

Avec $h^*(n)$: le coût réel depuis l'état n jusqu'à l'état final.

- Deux heuristiques admissibles h_1 et h_2 , on dit que h_1 **domine** h_2 si :

$$\forall n : h_1(n) \geq h_2(n)$$

- Une fonction heuristique est **consistante** si :

$$\forall n, \forall n' \in \{succ(n)\} : h(n) \leq C(n, n') + h(n')$$

Avec $C(n, n')$ le coût depuis l'état n jusqu'à l'état n'

Recherche meilleur d'abord

- Permet d'explorer les nœuds dans l'ordre (croissant/décroissant) de la valeur d'une fonction f .

Recherche meilleur d'abord

- Permet d'explorer les nœuds dans l'ordre (croissant/décroissant) de la valeur d'une fonction f .
- Deux cas particuliers existent :

Recherche meilleur d'abord

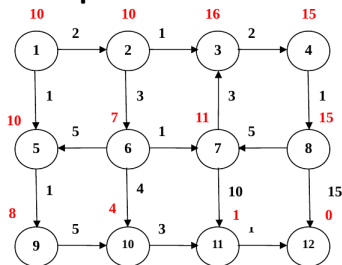
- Permet d'explorer les nœuds dans l'ordre (croissant/décroissant) de la valeur d'une fonction f .
- Deux cas particuliers existent :
 - ▶ L'algorithme de recherche **avare** (greedy, gourmand).

Recherche meilleur d'abord

- Permet d'explorer les nœuds dans l'ordre (croissant/décroissant) de la valeur d'une fonction f .
- Deux cas particuliers existent :
 - ▶ L'algorithme de recherche **avare** (greedy, gourmand).
 - ▶ l'algorithme **A*** (variantes : Iterative Deepening A* (IDA*), memory-bounded A* (MA*),...) .

La recherche avare (greedy search)

- **Principe** : L'objectif est de minimiser le coût estimé pour atteindre le but (la fonction h).
- Le nœud qui semble être le plus proche du but sera exploré en priorité.
- **Exemple** :



La recherche avare (greedy search)

Analyse de l'algorithme :

- **Complétude** : Complet, si l'espace de recherche fini et s'il y a absence de boucle ;

La recherche avare (greedy search)

Analyse de l'algorithme :

- **Complétude** : Complet, si l'espace de recherche fini et s'il y a absence de boucle ;
- **Complexité en temps** : exponentielle, $O(b^m)$.

La recherche avare (greedy search)

Analyse de l'algorithme :

- **Complétude** : Complet, si l'espace de recherche fini et s'il y a absence de boucle ;
- **Complexité en temps** : exponentielle, $O(b^m)$.
- **Complexité en espace** : exponentielle , $O(b^m)$.

La recherche avare (greedy search)

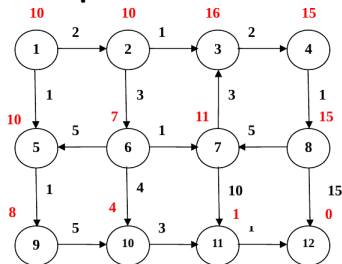
Analyse de l'algorithme :

- **Complétude** : Complet, si l'espace de recherche fini et s'il y a absence de boucle ;
- **Complexité en temps** : exponentielle, $O(b^m)$.
- **Complexité en espace** : exponentielle , $O(b^m)$.
- **Optimalité** : non.

l'algorithme A*

- **Principe** : L'idée est de combiner les deux méthodes de recherche, avare et coût uniforme.
- Chaque noeud n est évalué selon une fonction : $f(n) = g(n) + h(n)$.
 $g(n)$ est le coût de l'état initial au noeud n .
 $h(n)$ est un estimé du coût du noeud n à l'état final (heuristique).
- Le noeud qui minimise la fonction f sera visité en premier.

- **Exemple** :



l'algorithme A*

Analyse de l'algorithme :

- **Complétude** : Complet, si l'espace de recherche fini ;

l'algorithme A^*

Analyse de l'algorithme :

- **Complétude** : Complet, si l'espace de recherche fini ;
- **Complexité en temps** : exponentielle.

l'algorithme A*

Analyse de l'algorithme :

- **Complétude** : Complet, si l'espace de recherche fini ;
- **Complexité en temps** : exponentielle.
- **Complexité en espace** : exponentielle .

l'algorithme A*

Analyse de l'algorithme :

- **Complétude** : Complet, si l'espace de recherche fini ;
- **Complexité en temps** : exponentielle.
- **Complexité en espace** : exponentielle .
- **Optimalité** : oui, si h est admissible.

l'algorithme **A***

Quelques Propriétés :

- Si h est cohérente : h est aussi admissible.

l'algorithme A^*

Quelques Propriétés :

- Si h est cohérente : h est aussi admissible.
- Si h est cohérente : un noeud ne sera jamais revisité.

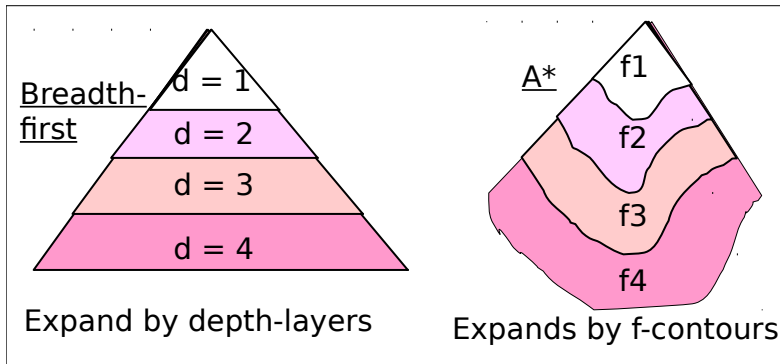
l'algorithme A^*

Quelques Propriétés :

- Si h est cohérente : h est aussi admissible.
- Si h est cohérente : un noeud ne sera jamais revisité.
- Si h n'est pas admissible et soit B la borne supérieur sur la surestimation du coût : $h(n) \leq h^*(n) + B$. A^* retourne une solution dont le coût est au plus B de plus que le coût optimal.

l'algorithme A*

Problème d'utilisation de la Mémoire :



Définition générique de f

- On peut définir la fonction f comme suit :

$$f(n) = (1 - w) \times g(n) + w * h(n)$$

Où w est un nombre réel : $0 \leq w \leq 1$.

Définition générique de f

- On peut définir la fonction f comme suit :

$$f(n) = (1 - w) \times g(n) + w * h(n)$$

Où w est un nombre réel : $0 \leq w \leq 1$.

- On obtient différents algorithmes de recherche selon la valeur de w :

Définition générique de f

- On peut définir la fonction f comme suit :

$$f(n) = (1 - w) \times g(n) + w * h(n)$$

Où w est un nombre réel : $0 \leq w \leq 1$.

- On obtient différents algorithmes de recherche selon la valeur de w :
 - ▶ $w = 0$: $f(n) = g(n)$, **recherche à coût uniforme**.

Définition générique de f

- On peut définir la fonction f comme suit :

$$f(n) = (1 - w) \times g(n) + w * h(n)$$

Où w est un nombre réel : $0 \leq w \leq 1$.

- On obtient différents algorithmes de recherche selon la valeur de w :
 - ▶ $w = 0$: $f(n) = g(n)$, **recherche à coût uniforme**.
 - ▶ $w = 1$: $f(n) = h(n)$, **recherche avare (greedy)**.

Définition générique de f

- On peut définir la fonction f comme suit :

$$f(n) = (1 - w) \times g(n) + w * h(n)$$

Où w est un nombre réel : $0 \leq w \leq 1$.

- On obtient différents algorithmes de recherche selon la valeur de w :
 - ▶ $w = 0$: $f(n) = g(n)$, **recherche à coût uniforme**.
 - ▶ $w = 1$: $f(n) = h(n)$, **recherche avare (greedy)**.
 - ▶ $w = 0.5$: $f(n) = g(n) + h(n)$, **A***.

- Artificial Intelligence : A Modern Approach, by Stuart Russell and Peter Norvig (Third Edition), chapitre II.
- Intelligent Systems, A Modern Approach, Crina Grosan and Ajith Abraham, chapitres 2 et 3.
- [http ://www.aishpace.org](http://www.aishpace.org).
- [https ://heuristicswiki.wikispaces.com/](https://heuristicswiki.wikispaces.com/)